

Personal Knowledge RAG Evaluation Sample

This document is intentionally longer than a normal quick demo note. With the current chunking settings of `chunk_size=500` and `overlap=50`, it should produce many chunks instead of only three. The goal is to make `top_k=3` meaningful: a retrieval result should select the relevant section from many possible chunks, not simply return the whole document.

Each section contains a unique anchor label. During manual evaluation, use the anchor label and nearby content to judge whether retrieval found the expected evidence. The anchor labels are not meant for end users; they exist to make retrieval evaluation easier and more reproducible.

Section 01 - Project Purpose [ANCHOR-PROJECT-PURPOSE]

The Personal Knowledge RAG Assistant is designed to help a user upload personal notes, split the notes into chunks, retrieve relevant context, and generate grounded answers. The project is not a generic chatbot. Its main value is that answers should be connected to uploaded source material. The system should make it clear which chunks were used as evidence, so a user can trace the answer back to the original document.

This project also serves as a portfolio project for AI application development. It demonstrates backend API design, document ingestion, chunk storage, embedding generation, retrieval ranking, prompt construction, and LLM provider boundaries. The goal is not to build a perfect enterprise system immediately. The goal is to build a credible RAG pipeline that can be explained clearly in interviews and improved step by step.

Section 02 - Upload Pipeline [ANCHOR-UPLOAD-PIPELINE]

The upload endpoint receives a file from the user. It first reads the file bytes and rejects empty files. Then it sends the file name and content to the ingestion layer, where text is

extracted from supported file types such as txt, markdown, and csv. Unsupported file types should produce a controlled client error instead of crashing the server.

After text extraction, the upload flow stores document metadata in SQLite. The metadata includes document_id, filename, extension, size_bytes, status, and created_at. The extracted text is then split into chunks. Each chunk has chunk_index, text, start_char, and end_char. These fields are important because later retrieval results need to point back to the original source position.

The upload endpoint then asks the embedding provider to create embeddings for each saved chunk. The resulting chunks contain an embedding field. The vector store boundary receives those embedded chunks and persists the embeddings. In the current local implementation, embeddings are stored in SQLite as JSON strings. In a future implementation, the same boundary can write vectors to Qdrant or another vector database.

Section 03 - Chunking Policy [ANCHOR-CHUNKING-POLICY]

The chunking module splits long text into smaller pieces. The current default chunk_size is 500 characters and the overlap is 50 characters. The overlap exists because important meaning can be located near a chunk boundary. Without overlap, a sentence or concept might be cut in half and retrieval quality could suffer.

The chunking function is intentionally narrow in responsibility. It should only split text into chunks and attach source character ranges. It should not call embedding models, write to the database, clean unsupported file types, or decide HTTP status codes. Keeping chunking narrow makes it easier to test and easier to reason about.

Chunk size has tradeoffs. If chunks are too large, retrieved context may include too much unrelated information and distract the model. If chunks are too small, useful meaning can be fragmented across several chunks. The current values are acceptable for learning, but real projects should evaluate chunk size against actual documents and retrieval results.

Section 04 - Storage Layer [ANCHOR-STORAGE-LAYER]

The storage layer owns SQLite persistence. It creates tables, inserts documents, inserts chunks, loads chunks by document_id, and updates chunk embeddings. The documents table stores document-level metadata. The chunks table stores chunk-level text and source ranges. The embedding_json field stores a serialized embedding vector.

The function get_document is used by the API layer to check whether a document exists. If a document_id is missing, the retrieve endpoint can return a controlled 404 response. The function get_chunks_by_document loads all chunks for a document and converts embedding_json back into a Python list of floats. This conversion is important because retrieval code expects embeddings as list[float], not raw JSON strings.

The storage layer should not decide ranking logic. It should not compute cosine similarity, call an LLM, or decide how an answer should be phrased. It is responsible for persistence and data shape, not retrieval strategy.

Section 05 - Embedding Provider Boundary [ANCHOR-EMBEDDING-PROVIDER]

The embedding provider boundary hides the details of converting text into vectors. The same upload and retrieval flows should work whether the project uses fake embeddings, a local BGE model, or a future remote embedding API. The abstract provider exposes embed_text for one piece of text and embed_chunks for a list of chunks.

FakeEmbeddingProvider exists for unit tests and CI. It creates deterministic vectors using hashing. It does not represent real semantic similarity, but it makes tests stable and fast. LocalEmbeddingProvider uses a sentence-transformers model such as BAAI bge-small-zh-v1.5. It produces real semantic embeddings and is suitable for local demo and retrieval quality checks.

The local provider should not reload the BGE model on every upload or retrieve request. Loading a model includes reading model files, loading tokenizer data, initializing neural network weights, and occupying memory. A provider cache can keep one loaded provider in memory, while a reset function can clear the cache during tests.

Section 06 - Vector Store Boundary [ANCHOR-VECTOR-STORE]

The vector store boundary hides how embeddings are persisted and searched. Current code can use SQLiteVectorStore, which loads chunks from SQLite and ranks them with cosine similarity. Future code can add QdrantVectorStore without changing the API endpoint shape. The API should ask for a vector store through `get_vector_store` rather than directly importing a concrete implementation everywhere.

VectorStore should receive `query_embedding` rather than raw question text. Text-to-vector conversion belongs to the embedding provider. VectorStore is responsible for vector persistence and vector search. This separation prevents one module from doing too many jobs.

The current search method still accepts `document_id`, which is reasonable for a single-document learning demo. However, real knowledge bases often need metadata filters, such as `workspace_id`, `user_id`, `document_type`, `tags`, `source`, or `created_at` range. This project plans to add a small metadata filter step later, but it will not implement a full enterprise permission system in this phase.

Section 07 - Retrieval Ranking [ANCHOR-RETRIEVAL-RANKING]

The retrieval process embeds the user question and compares the query embedding with stored chunk embeddings. The ranking function uses cosine similarity. A higher score means the query vector and chunk vector point in a more similar direction. The result is a list of chunks sorted by score, with the highest scoring chunks returned first.

Each chunk embedding represents the whole chunk, not every character inside the chunk. For a BGE model, a chunk of several hundred characters can be encoded into one fixed-size vector such as 512 dimensions. Internally, the model tokenizes the text, runs transformer layers, and pools token representations into one chunk-level embedding.

The current dense retrieval approach is simple and useful, but not perfect. Sometimes the correct chunk appears in top-3 but not top-1. That situation is a good candidate for reranking. Sometimes the correct chunk does not appear at all. That may require better chunking, better documents, better embeddings, metadata filters, or query rewriting.

Section 08 - Grounded Prompt Builder [ANCHOR-PROMPT-BUILDER]

The prompt builder converts retrieved chunks into a grounded prompt for the LLM. It should clearly separate source context from the user question. It should also instruct the model to answer only from the provided context and avoid inventing unsupported facts. This reduces hallucination risk, although it does not eliminate it completely.

The prompt should preserve source information such as `chunk_id`, `chunk_index`, and text snippets. The answer generation layer can then return sources together with the final answer. Source transparency is important because RAG users need to verify whether the answer is grounded in actual retrieved evidence.

Prompt building should not perform retrieval itself. It should receive matches that were already retrieved. This keeps retrieval, prompt construction, and LLM generation as separate responsibilities.

Section 09 - LLM Client Boundary [ANCHOR-LLM-CLIENT]

The LLM client boundary hides model provider details. The project can use a fake LLM client for tests, a failing fake client for error handling tests, and an Ollama client for local model calls. The answer generation flow should not know whether the underlying

provider is fake or real. It should only call a common generate method.

The Ollama client sends a POST request to the local Ollama API. It uses a non-streaming call so the API layer can receive one complete answer at a time. If the provider times out, returns an HTTP error, or returns an empty answer, the client should raise an `LLMProviderError`. The API layer can then convert that provider error into a controlled HTTP response.

Provider errors should not leak private details or raw stack traces to the user. A stable API should return a clear error message and status code while keeping internal implementation details protected.

Section 10 - Answer Endpoint [ANCHOR-ANSWER-ENDPOINT]

The answer endpoint receives a `document_id`, a question, and `top_k`. It first reuses retrieval to obtain relevant chunks. Then it builds a grounded prompt from the retrieved matches. Finally, it calls the LLM client and returns an answer, provider name, sources, and confidence notes.

The answer endpoint should not duplicate retrieval logic. If retrieval behavior changes, the answer endpoint should benefit from that change automatically. This is why answer calls `retrieve` internally or reuses the same retrieval service path. Duplicated retrieval logic would make the project harder to maintain.

The answer response should include sources because source visibility is central to a RAG application. Without sources, the user cannot easily inspect whether the model answer is actually grounded.

Section 11 - Configuration and Environment [ANCHOR-CONFIG-ENV]

The project uses a `Settings` object to centralize configuration. The `.env` file provides

concrete local values. The `config.py` module reads those values and exposes a shared settings object. Application modules import settings instead of reading environment variables directly.

This pattern makes configuration easier to control. For example, `EMBEDDING_PROVIDER` can be `fake` for tests or `local` for demo. `LLM_PROVIDER` can be `fake`, `failing_fake`, or `ollama`. `VECTOR_STORE_PROVIDER` can be `sqlite` now and `qdrant` later. Tests often patch the shared settings object directly with `monkeypatch.setattr` because settings is already created at import time.

The `.env.example` file documents expected configuration names but should not contain private secrets. It is a template for other developers, not the real local configuration file.

Section 12 - Testing and CI [ANCHOR-TESTING-CI]

Tests should remain stable in GitHub Actions. CI should not depend on local BGE model files, a running Ollama service, or a local Qdrant container. Unit tests should use fake providers unless they are explicitly marked as integration tests. This keeps the project reproducible for reviewers and avoids false failures.

When provider caches are added, tests need a way to reset those caches. Otherwise one test may configure a local provider and another test may unexpectedly reuse it. A `reset_embedding_provider_cache` function and an `autouse` fixture in tests can keep tests isolated.

Good tests should check behavior and data shape. They should avoid hardcoding fragile real-model retrieval rankings in ordinary CI. Manual evaluation can record real retrieval quality in markdown files, while CI checks basic structure and core deterministic behavior.

Section 13 - Retrieval Evaluation [ANCHOR-RETRIEVAL-EVALUATION]

Retrieval evaluation records whether the system found the expected evidence. A useful evaluation question should have an expected source section, expected evidence, and failure signal. Results should track top-1 hit, top-3 hit, status, and notes. Status can be pass, partial, or fail.

Pass means the best returned chunk directly contains the expected evidence. Partial means the expected evidence appears in top-3 but not as the best match, or the evidence is incomplete. Fail means the expected evidence is not found in top-3. Partial cases are especially useful because reranking may improve them.

Evaluation should be honest. The goal is not to make every row pass. The goal is to understand which parts of the RAG system work and which parts need improvement.

Section 14 - Reranking Plan [ANCHOR-RERANKING-PLAN]

Reranking is planned as a separate boundary after vector retrieval. The vector store can retrieve more candidates, such as top_k=10. A reranker can then reorder those candidates and return a smaller final set, such as top_n=3. This can improve cases where the correct chunk appears in top-3 or top-10 but not at rank one.

The first reranker should be fake or rule-based so tests remain deterministic. A real reranker model can be added later, but it should not be required for normal CI. The purpose of the first reranker day is to create the boundary and understand where reranking fits in the pipeline.

Reranking should not replace evaluation. It should be judged against the retrieval baseline. If the baseline does not show partial failures, reranking may not have a clear measurable benefit.

Section 15 - Qdrant Plan [ANCHOR-QDRANT-PLAN]

Qdrant is a vector database that can store vectors and perform similarity search efficiently. A minimal local Qdrant integration can run through Docker. The project can add QdrantVectorStore behind the existing VectorStore boundary while keeping SQLite as the default provider for tests and CI.

The first Qdrant integration should be intentionally small. It should create or use one collection, upsert vectors for chunks, and search vectors by query embedding. It should not attempt production deployment, advanced replication, cloud credentials, or complex multi-tenant permissions on the first day.

Qdrant is useful for demonstrating awareness of real vector database architecture. However, the project should still keep a SQLite fallback so that ordinary tests remain simple and stable.

Section 16 - Metadata Filter Plan [ANCHOR-METADATA-FILTER]

Single document_id retrieval is useful for learning, but it is limited. Real knowledge bases often search across many documents and narrow results using metadata filters. Metadata can include source, document_type, tags, workspace, user, or created_at.

This project plans to add a small metadata filter step after the vector store boundary is stable. The goal is not to build a full enterprise permission system. The goal is to show how retrieval can move from one-document search toward knowledge-base search.

Metadata filters help answer questions such as: search only policy documents, search only notes tagged with interview, or search only documents from one workspace. This is closer to real RAG applications in companies.

Section 17 - Project Limitations [ANCHOR-PROJECT-LIMITATIONS]

The current project is a learning and portfolio RAG system, not a production enterprise

platform. It does not yet implement full authentication, user-level permissions, production monitoring, large-scale ingestion queues, PDF and Word parsing, or multi-tenant data isolation.

The project currently focuses on the most interview-relevant parts of an AI application: ingestion, chunking, embeddings, vector store boundary, retrieval, prompt building, LLM client boundary, evaluation, and planned reranking. These pieces show practical AI application engineering without pretending that every production concern is solved.

The honest limitation statement is important. A credible engineer can explain both what the system does and what it does not do yet.